AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

EXERCISE 01 · UNIT 1

Performance metrics and the roofline, in code

Implement every formula from Unit 1. The marks are in the edge cases, which is exactly where real implementations of these go wrong.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

EXERCISE	UNIT
ex01-performance-metrics	1 — Architecture and performance
LANGUAGE	MARKS
C	10
SUBMIT AS	OFFERING
<ROLLNO>.c	Odd Semester 2026-27

What you are building

Eleven small functions, declared in `include/metrics.h`. Every formula is in the Unit 1 notes, so the arithmetic is not the difficulty.

FUNCTION	FORMULA
`speedup(t1, tN)`	$S = T_1 / T_N$
`efficiency(t1, tN, N)`	$E = S / N$
`amdahl(s, N)`	$S = 1 / (s + (1-s)/N)$
`amdahl_limit(s)`	$1 / s$
`gustafson(s, N)`	$S = N - s(N-1)$
`karp_flatt(S, N)`	$e = (1/S - 1/N) / (1 - 1/N)$
`arithmetic_intensity(flops, bytes)`	$I = W / Q$
`ridge_point(peak, bw)`	P_{peak} / β
`attainable_gflops(peak, bw, I)`	$\min(P_{peak}, \beta \cdot I)$
`is_memory_bound(peak, bw, I)`	$I <$ ridge
`roofline_fraction(peak, bw, I, achieved)`	achieved / attainable

You will use these for the rest of the course: in your labs, your assignments, and your mini project. Writing them once, correctly, is worth the hour.

Where the marks actually are

The formulas take ten minutes. The hidden tests are about what happens at the boundaries, and every one of these is a decision that real code has to make:

PITFALL · THE QUESTIONS THE HEADER ANSWERS, AND THE TESTS CHECK

- What is the speedup when T_N is zero or negative? (Return 0, do not divide by zero.)
- What is Amdahl's ceiling when $s = 0$? (There is none. Return −1 to say so, rather than dividing by zero or returning a huge number.)
- What is Karp-Flatt on **one** processor? (Undefined: the formula divides by $1 - 1/1$. Return −1.)
- Is a kernel sitting **exactly** at the ridge point memory bound or compute bound? (The header defines it as compute bound, so your comparison must be strict.)
- Should `roofline_fraction` divide by peak or by **attainable**? (Attainable. Dividing by peak is the mistake that makes every memory-bound kernel look like a failure.)

Read `include/metrics.h` line by line. It states the required behaviour for every one of these.

COMMON MISCONCEPTION · SUPERLINEAR SPEEDUP SHOULD BE CLAMPED TO N

It should not. $S > N$ is a real, explainable effect: when the data is split across N processors, each piece may now fit in cache where the whole did not. Your `speedup` and `efficiency` must report it faithfully. A hidden test checks that `efficiency(46, 10, 4)` returns 1.15 and not 1.0.

The worked example this makes concrete

One hidden test walks through the decision from the Unit 1 notes end to end:

```
double I = arithmetic_intensity(2.0, 16.0);       /* dot product: 0.125    */
ridge_point(3000.0, 200.0);                       /* 15 FLOP/byte          */
is_memory_bound(3000.0, 200.0, I);                /* 1: far left of ridge  */
attainable_gflops(3000.0, 200.0, I);              /* 25 GFLOP/s            */
roofline_fraction(3000.0, 200.0, I, 22.0);        /* 0.88, not 0.007       */
```

That last line is the whole point of the exercise. Twenty-two GFLOP/s is 0.7 percent of peak, which sounds like a disaster, and **88 percent of what the machine can actually deliver for this kernel**, which means stop optimising.

Building and testing

```
./selfcheck.sh
```

How this is marked

CHECK	MARKS
Compiles cleanly with warnings as errors	1
Public tests: the formulas on ordinary inputs	4
Hidden tests: edge cases and degenerate inputs	5

Half the marks are in the hidden set, deliberately.

Submitting

```
AIE23001.c
```

One file. No `main()`.